DBMS TASK 3

Relational Structure Review and Normalization

Purpose

This report reviews the structure of OnlineShoppingDB, identifies dependencies between fields, and explains how separating business entities helps prevent duplicate data and common database anomalies.

A. Entity Map and Key Dependencies

Entity / Table	Identifier	Dependency Rule
Customers	CustomerID	CustomerID determines CustomerName, Email, Phone, and Address. Email must remain unique.
Products	ProductID	ProductID determines ProductName, Category, Price, and Stock.
Orders	OrderID	OrderID determines CustomerID, OrderDate, and Status.
OrderDetails	OrderDetailID	OrderDetailID determines OrderID, ProductID, Quantity, and UnitPrice.
Payments	PaymentID	PaymentID determines OrderID, PaymentMethod, Amount, and PaymentStatus.

B. Relationship Overview

Parent	Relationship	Child
Customers	One customer can place multiple orders	Orders
Orders	One order can contain multiple line items	OrderDetails
Products	One product may occur in many order lines	OrderDetails
Orders	An order can have one or more payment records	Payments

C. Where Poor Design Causes Problems

Issue	Possible Scenario	Design Improvement
Repeated data	Customer contact details are copied into every order row.	Keep customer information in Customers and reference it by CustomerID.
Insertion problem	A product cannot be listed until an order is created.	Store products separately from sales transactions.
Update problem	A changed customer address must be edited in several rows.	Maintain one customer record and update it in one place.
Deletion problem	Removing an order might also remove the only copy of a product's details.	Keep product details independent from OrderDetails.

D. Normalization Decisions

First Normal Form (1NF): each column contains a single value and every record is uniquely identified. A separate OrderDetails row is used for each product in an order.

Second Normal Form (2NF): the design avoids partial dependencies by using a single-column primary key in each table. Product descriptions are kept in Products rather than repeated in order lines.

Third Normal Form (3NF): non-key attributes describe their table's key. Customer contact fields depend on CustomerID, product attributes depend on ProductID, and order attributes depend on OrderID.

UnitPrice in OrderDetails stores the price charged on a specific purchase. Products.Price represents the current catalogue price, so historical order values remain meaningful.

E. SQL Report: Order Line Details

```
SELECT o.OrderID, c.CustomerName, p.ProductName,
       d.Quantity, d.UnitPrice,
       d.Quantity * d.UnitPrice AS LineTotal
FROM Orders AS o
INNER JOIN Customers AS c ON c.CustomerID = o.CustomerID
INNER JOIN OrderDetails AS d ON d.OrderID = o.OrderID
INNER JOIN Products AS p ON p.ProductID = d.ProductID
ORDER BY o.OrderID, d.OrderDetailID;
```

Run the query against your own database and place a screenshot of the result grid in this section. The sample below illustrates the expected shape of the report.

Order	Customer	Item	Qty	Price	Line Total
1	Arun Prabu	Laptop	1	55000.00	55000.00
1	Arun Prabu	Headphones	2	1500.00	3000.00
2	Rahul Kumar	Smartphone	1	25000.00	25000.00
2	Rahul Kumar	Keyboard	2	800.00	1600.00
3	Priya Sharma	Headphones	1	1500.00	1500.00
4	Arun Prabu	Office Chair	1	4500.00	4500.00
5	Sneha Reddy	Smartphone	1	25000.00	25000.00
6	Karthik Raj	Keyboard	1	800.00	800.00

Closing Remarks

Separating customers, products, orders, order lines, and payments keeps each fact in an appropriate table. Keys and relationships reduce duplication and make accurate joined reports possible.

SQL CODE — MYSQL WORKBENCH

Run after creating the tables and inserting sample records.

```
USE OnlineShoppingDB;

SELECT o.OrderID, c.CustomerName, p.ProductName,
       d.Quantity, d.UnitPrice,
       d.Quantity * d.UnitPrice AS LineTotal
FROM Orders o
JOIN Customers c ON c.CustomerID = o.CustomerID
JOIN OrderDetails d ON d.OrderID = o.OrderID
JOIN Products p ON p.ProductID = d.ProductID
ORDER BY o.OrderID, d.OrderDetailID;
```